What is a Promise in JavaScript?

A **Promise** is a JavaScript object that represents the **future result** of an asynchronous operation.

Think of it like this:

Promise = "I promise I will give you the result later."

For example:

- Downloading data from an API
- Reading a file
- Waiting for a timer

These tasks take time, so JavaScript doesn't wait. Instead, it continues executing other code and uses a **Promise** to notify you when the task is complete.

Real-Life Example

Imagine you order pizza.

1. You place the order.
2. The restaurant starts preparing it.
3. You don't stand there waiting—you do other work.
4. Later:
    - Pizza arrives (Success)
    - Or the restaurant cancels (Failure)

This is exactly how a Promise works.

Promise States

A Promise has **3 states**.

State	Meaning
Pending	Work is still in progress
Fulfilled	Task completed successfully
Rejected	Task failed

```
Pending
  |
  +------> Fulfilled ✅
  |
  +------> Rejected ❌
```

Syntax

```
let promise = new Promise(function(resolve, reject) {

   // asynchronous work

});
```

- resolve() → Success
- reject() → Failure

Example 1: Simple Promise

```
let promise = new Promise(function(resolve, reject) {
   let success = true;
   if(success){
      resolve("Data Loaded Successfully");
   }else{
      reject("Something Went Wrong");
   }
});
promise
.then(function(result){
   console.log(result);
})
.catch(function(error){
   console.log(error);
});
```

Output
Data Loaded Successfully

Example 2: Promise with setTimeout()

```
let promise = new Promise(function(resolve, reject){

   setTimeout(function(){

      resolve("Order Delivered");

   },3000);

});

promise.then(function(data){

   console.log(data);

});
```

Output (after 3 seconds)
Order Delivered

Example 3: Reject Example

```
let promise = new Promise(function(resolve, reject){

   let internet = false;

   if(internet){
      resolve("File Downloaded");
   }else{
      reject("No Internet Connection");
```

```
    }

});

promise
.then(function(data){

    console.log(data);

})
.catch(function(error){

    console.log(error);

});
```

Output

No Internet Connection

---

then()

Used when the Promise is successful.

```
promise.then(function(result){

    console.log(result);

});
```

---

catch()

Used when the Promise fails.

```
promise.catch(function(error){

    console.log(error);
```

```
});
```

---

finally()

Runs whether the Promise succeeds or fails.

```
promise
.then(function(result){

   console.log(result);

})
.catch(function(error){

   console.log(error);

})
.finally(function(){

   console.log("Process Finished");

});
```

Output

```
Data Loaded Successfully
Process Finished
or
Something Went Wrong
Process Finished
```

Promise Flow

```
Create Promise
     |
```

```
      |
      V
Pending
      |
      |
----------------------
|                    |
Success            Failure
|                    |
resolve()          reject()
|                    |
then()             catch()
        |
        |
   finally()
```

Example 4: Fetching Data (API)

```
fetch("https://jsonplaceholder.typicode.com/users")
.then(function(response){

   return response.json();

})
.then(function(data){

   console.log(data);

})
.catch(function(error){

   console.log(error);
```

});
Here:

- fetch() returns a Promise.
- response.json() also returns a Promise.
- The second then() receives the actual data.

Promise Chaining

```
let promise = new Promise(function(resolve){

   resolve(10);

});

promise
.then(function(num){

   return num + 10;

})
.then(function(num){

   return num * 2;

})
.then(function(num){

   console.log(num);

});
```

Output

Why Do We Need Promises?

Without Promises, asynchronous code often leads to deeply nested callbacks, commonly called **"Callback Hell"**.

Without Promise

```
login(function(){

   getProfile(function(){

      getPosts(function(){

         getComments(function(){

            console.log("Done");

         });

      });

   });

});
```

This becomes difficult to read and maintain.

With Promise

```
login()
.then(getProfile)
.then(getPosts)
.then(getComments)
.then(function(){
```

```
    console.log("Done");

})
.catch(function(error){

    console.log(error);

});
```

This is cleaner and easier to understand.

Promise vs Callback

Callback	Promise
Difficult to manage	Easy to manage
Callback Hell	Clean code
Hard error handling	Easy with catch()
Less readable	More readable
Old approach	Modern approach

Interview Questions

1. What is a Promise?

A Promise is an object that represents the eventual success or failure of an asynchronous operation.

---

2. How many states does a Promise have?

- Pending
- Fulfilled
- Rejected

---

3. What does resolve() do?

It marks the Promise as successful and passes the result to then().

---

4. What does reject() do?
It marks the Promise as failed and passes the error to catch().

---

5. What is the difference between then() and catch()?

then()	catch()
Runs on success	Runs on failure

Quick Summary
Promise

↓

Pending

↓

resolve() ----------> then()

OR

reject() ----------> catch()

↓

finally()
Remember this simple rule:

- **new Promise()** → Create a promise.
- **resolve()** → Success.

- **reject()** → Failure.
- **.then()** → Handle success.
- **.catch()** → Handle errors.
- **.finally()** → Runs every time.

This is the foundation for modern asynchronous JavaScript and is also the basis for **async/await**, which provides an even cleaner way to work with Promises.

Async/Await in JavaScript

async/await is a modern way to work with **Promises**. It makes asynchronous code look like normal synchronous code, making it easier to read and maintain.

Syntax

```
async function functionName() {

   let result = await promise;

}
```

Keywords

- **async** → Makes a function return a Promise.
- **await** → Waits for a Promise to complete (can only be used inside an async function).

Example 1: Basic Async/Await

```
function myPromise() {

   return new Promise(function(resolve) {

      setTimeout(function() {
         resolve("Hello Students!");
      }, 2000);
```

```
    });
}

async function showMessage() {
    console.log("Loading...");
    let result = await myPromise();
    console.log(result);
    console.log("Program Finished");
}

showMessage();
```

Output

```
Loading...
(Wait 2 seconds)

Hello Students!
Program Finished
```

Example 2: Student Result

```
function checkResult(marks) {
    return new Promise(function(resolve, reject) {
        if (marks >= 33) {
```

```
            resolve("Pass");
        } else {
            reject("Fail");
        }

    });

}

async function result() {

    try {

        let data = await checkResult(80);
        console.log(data);

    }
    catch(error) {

        console.log(error);

    }

}

result();
```

Output

Pass

If you call:

```
checkResult(20);
```

Output

Fail

Example 3: Fetch API with Async/Await

```
async function getUsers() {

   try {

      let response = await
fetch("https://jsonplaceholder.typicode.com/users");

      let data = await response.json();

      console.log(data);

   }
   catch(error) {

      console.log("Error:", error);

   }

}

getUsers();
```

Explanation

1. fetch() sends a request to the server.
2. await waits for the response.
3. response.json() converts the response to JavaScript objects.
4. The data is printed to the console.

Example 4: Delay Program

```
function delay() {

   return new Promise(function(resolve) {

      setTimeout(function() {
         resolve();
      }, 3000);

   });

}

async function start() {

   console.log("Start");

   await delay();

   console.log("3 Seconds Completed");

}

start();
```

Output

```
Start

(Wait 3 seconds)

3 Seconds Completed
```

Example 5: Multiple Await

```
function task1() {

   return new Promise(function(resolve) {

      setTimeout(function() {
         resolve("Task 1 Completed");
      }, 2000);

   });

}

function task2() {

   return new Promise(function(resolve) {

      setTimeout(function() {
         resolve("Task 2 Completed");
      }, 1000);

   });

}

async function executeTasks() {

   let a = await task1();
   console.log(a);

   let b = await task2();
```

```
    console.log(b);

}

executeTasks();
```

Output

(After 2 seconds)

Task 1 Completed

(After 1 more second)

Task 2 Completed

Example 6: Real-Life Login Simulation

```
function login(username, password) {

    return new Promise(function(resolve, reject) {

        setTimeout(function() {

            if (username === "admin" && password === "1234") {
                resolve("Login Successful");
            } else {
                reject("Invalid Username or Password");
            }

        }, 2000);

    });
```

```
}

async function userLogin() {

   try {

      let message = await login("admin", "1234");

      console.log(message);

   }
   catch(error) {

      console.log(error);

   }

}

userLogin();
```

Output

Login Successful

Error Handling with try...catch

```
async function example() {

   try {

      let result = await Promise.reject("Something went
wrong");
```

```
        console.log(result);

    }
    catch(error) {

        console.log(error);

    }

}

example();
```

Output

Something went wrong

Async/Await Flow

Start
  |
  ▼
Call async function
  |
  ▼
await Promise
  |
  ▼
Promise Completed
  |
  ▼
Next Line Executes
  |
  ▼
End

Promise vs Async/Await

Promise (.then())	Async/Await
Uses .then()	Uses await
Uses .catch()	Uses try...catch
Can become harder to read with many .then() calls	Cleaner and easier to read
Good for chaining	Best for sequential asynchronous code

Promise

```
fetch(url)
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.log(error));
```

Async/Await

```
async function getData() {

   try {

      let response = await fetch(url);
      let data = await response.json();

      console.log(data);

   }
   catch(error) {

      console.log(error);

   }
```

}

Interview Questions

1. What is async?

async is a keyword that makes a function return a Promise.

---

2. What is await?

await pauses the execution of an async function until the Promise is resolved or rejected.

---

3. Can await be used outside an async function?

No. (Except in top-level await in JavaScript modules.)

---

4. Why do we use try...catch with async/await?

To handle errors from rejected Promises.

---

5. Does await block the entire JavaScript program?

No. It pauses only the current async function. The rest of the JavaScript program can continue running.

Easy Formula to Remember

Promise
↓
async function()
↓
await Promise
↓
Result
↓
try...catch (Error Handling)